

# PYTHON SOURCE CODE DOCUMENTATION

*Virtual Pet Simulator*

---

---

Assignment 2

VaultofCode Python Programming Internship

**Submitted By:** Kanak Chaudhary

**Course:** Bachelor of Computer Applications (BCA)

**Role:** Python Programming Intern

## About This Document

---

This document presents the complete Python source code developed for the Virtual Pet Simulator assignment, undertaken as part of the VaultofCode Python Programming Internship. It serves as a standalone technical reference intended to accompany the main project report, focusing exclusively on the implementation rather than on project background or output discussion.

The program was designed and structured using sound programming principles appropriate for an interactive, menu-driven console application. Its implementation demonstrates a range of core Python concepts, including:

- Functions
- Dictionaries
- Loops
- Conditional Statements
- User Input Handling
- Modular Programming
- Clean Coding Practices

The sections that follow present the source code in its complete and unmodified form, formatted for clarity and ease of review.

# Complete Python Source Code

---

*File: solution.py*

```
# =====
# Virtual Pet Simulator
# Assignment 2 - VaultofCode Internship
# Developed by: Kanak Chaudhary
# =====

def show_menu():
    print("\n" + "=" * 45)
    print("          VIRTUAL PET SIMULATOR")
    print("=" * 45)
    print("1. Feed Pet")
    print("2. Play with Pet")
    print("3. Check Pet Status")
    print("4. Exit")
    print("=" * 45)

def feed_pet(pet):

    print(f"\n You fed {pet['name']}.")

    pet["hunger"] -= 10
    pet["happiness"] += 5
    pet["health"] += 2
    pet["days"] += 1

    pet["hunger"] = max(0, pet["hunger"])
    pet["happiness"] = min(100, pet["happiness"])
    pet["health"] = min(100, pet["health"])

def play_with_pet(pet):

    print(f"\n You played with {pet['name']}.")

    pet["happiness"] += 15
    pet["hunger"] += 10
    pet["health"] += 3
    pet["days"] += 1

    pet["happiness"] = min(100, pet["happiness"])
    pet["hunger"] = min(100, pet["hunger"])
    pet["health"] = min(100, pet["health"])

def check_status(pet):

    if pet["hunger"] > 80:
        mood = "\n Very Hungry"

    elif pet["happiness"] >= 70:
        mood = "\n Happy"
```

```

elif pet["happiness"] >= 40:
    mood = "😊 Normal"

else:
    mood = "😞 Sad"

print("\n" + "=" * 40)
print("          PET STATUS")
print("=" * 40)
print(f"🐾 Pet Name      : {pet['name']}")
print(f"🐾 Day           : {pet['days']}")
print(f"🐾 Health        : {pet['health']}%")
print(f"🐾 Happiness     : {pet['happiness']}%")
print(f"🐾 Hunger        : {pet['hunger']}%")
print(f"🐾 Mood          : {mood}")
print("=" * 40)

def update_pet(pet):
    """
    Automatically updates the pet's condition
    after every user action.
    """

    # Time passes
    pet["hunger"] += 5
    pet["happiness"] -= 2

    # Keep values within limits
    pet["hunger"] = min(100, pet["hunger"])
    pet["happiness"] = max(0, pet["happiness"])

    # If hunger becomes too high
    if pet["hunger"] > 80:
        print(f"\n🐾 {pet['name']} is getting very hungry!")

        pet["happiness"] -= 10

        if pet["happiness"] < 0:
            pet["happiness"] = 0

def game_over(pet):

    if pet["health"] <= 0:
        print("\nYour pet became unhealthy.")
        print("Game Over!")
        return True

    if pet["hunger"] >= 100:
        print(f"\n🐾 {pet['name']} became too hungry.")
        print("Game Over!")
        return True

    if pet["happiness"] <= 0:
        print("\nYour pet became very sad.")
        print("Game Over!")
        return True

```

```

    return False

def show_summary(pet):

    print("\n" + "=" * 45)
    print("          GAME SUMMARY")
    print("=" * 45)

    print(f"Pet Name       : {pet['name']}")
    print(f"Days Played    : {pet['days']}")
    print(f"Final Health   : {pet['health']}%")
    print(f"Final Happiness : {pet['happiness']}%")
    print(f"Final Hunger   : {pet['hunger']}%")

    score = pet["health"] + pet["happiness"] - pet["hunger"]

    print(f"Final Score    : {score}")

    print("=" * 45)

def main():

    print("=" * 45)
    print("          WELCOME TO VIRTUAL PET SIMULATOR")
    print("=" * 45)

    pet_name = input("Enter your pet's name: ")

    pet = {
        "name": pet_name,
        "happiness": 50,
        "hunger": 50,
        "health": 100,
        "days": 1
    }

    print(f"\nWelcome, {pet['name']}!")
    print("Take good care of your virtual pet.")

    while True:

        show_menu()

        choice = input("Enter your choice (1-4): ")

        if choice == "1":

            feed_pet(pet)

            update_pet(pet)

            check_status(pet)

            if game_over(pet):
                break

        elif choice == "2":

```

```
        play_with_pet(pet)
        update_pet(pet)
        check_status(pet)
        if game_over(pet):
            break

    elif choice == "3":
        check_status(pet)

    elif choice == "4":
        show_summary(pet)

        print(f"\nThank you for taking care of {pet['name']}!")
        print("See you again!")

        break

    else:
        print("\nInvalid choice!")
        print("Please enter a number between 1 and 4.")

if __name__ == "__main__":
    main()
```

## Documentation Notes

---

The source code presented in this document has been developed in accordance with clean and disciplined programming practices suitable for academic evaluation. Each major action within the Virtual Pet Simulator — feeding, playing, checking status, updating pet conditions, and generating a summary — is implemented as an independent function, ensuring that the program remains modular, readable, and easy to maintain.

This functional separation allows individual components of the program to be understood, tested, and modified without affecting unrelated parts of the codebase. Dictionaries are used to represent the pet's state in a structured manner, while loops and conditional statements govern the program's interactive flow and decision-making logic. Meaningful comments have been included throughout the code to improve readability and long-term maintainability, allowing future reviewers to quickly understand the purpose of each function and code block.

Overall, the implementation satisfies the requirements outlined for Assignment 2 of the VaultofCode Python Programming Internship, reflecting a sound understanding of core Python programming concepts including functions, dictionaries, loops, conditional statements, and user input handling.

This document is intended to accompany the corresponding project report and contains the complete implementation of the assignment for the purpose of academic evaluation.